Iris Dataset Analysis

Prepared for: Dr. Vineetha Menon
Prepared by: Azaria A. Reed
Date: February 22, 2026

Data Visualization

Correlation Coefficient Matrix

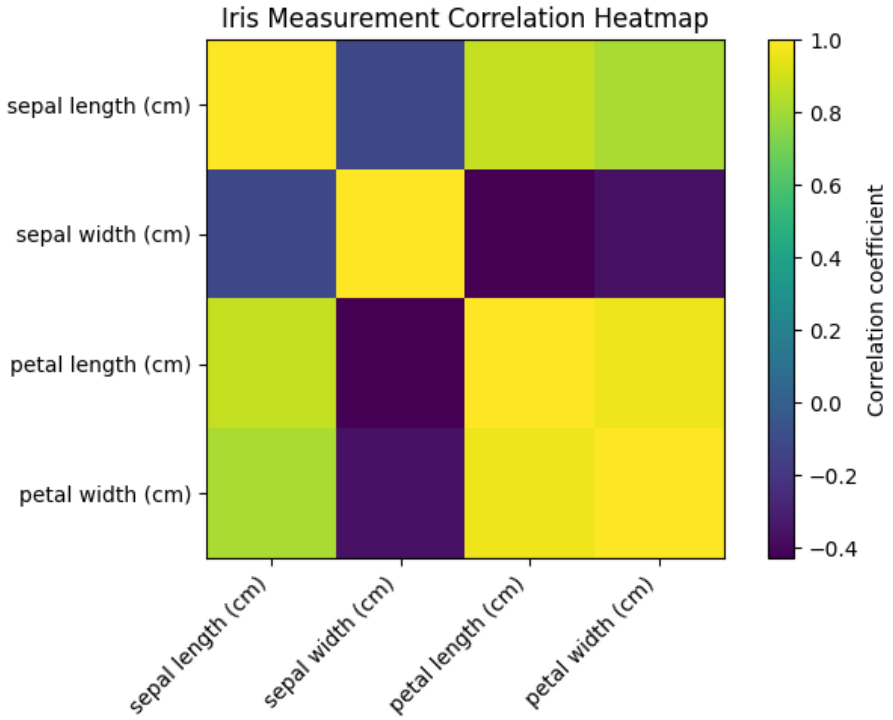

Figure 1. Iris Measurement Correlation Heatmap.

Feature Analysis Plots

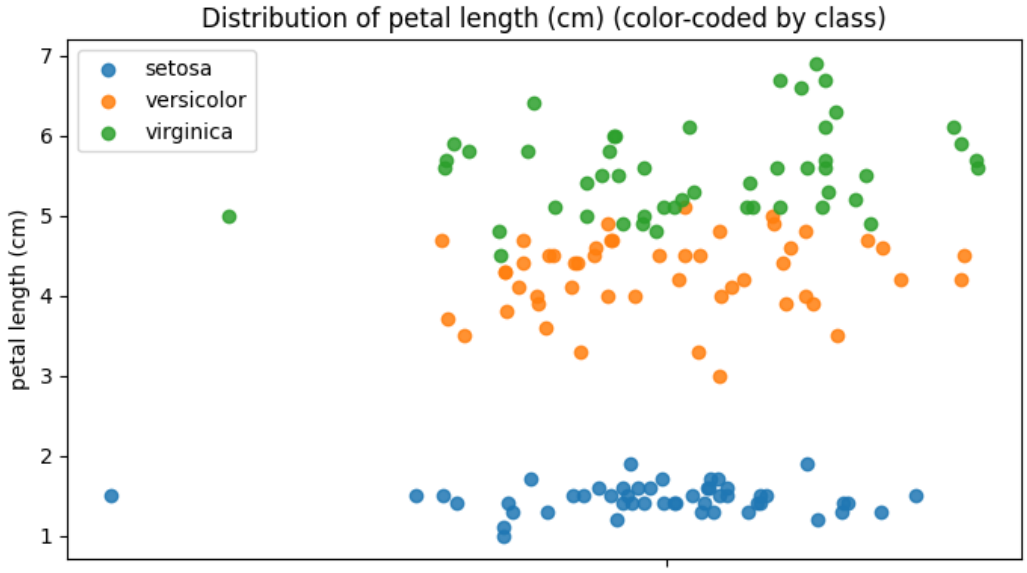

Figure 2. Distribution of Petal Length (cm), color-coded by class.

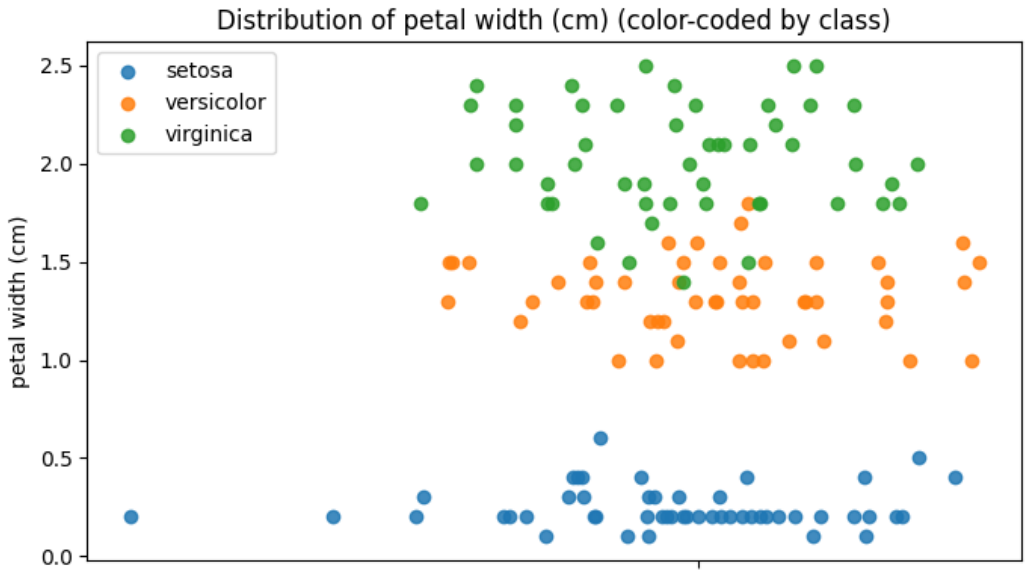

Figure 3. Distribution of Petal Width (cm), color-coded by class.

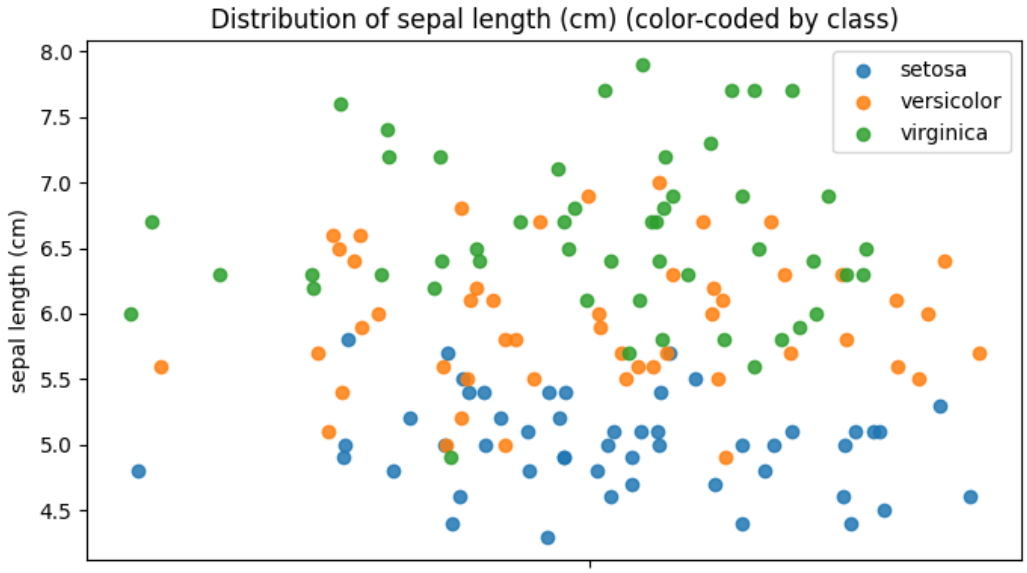

Figure 4. Distribution of Sepal Length (cm), color-coded by class.

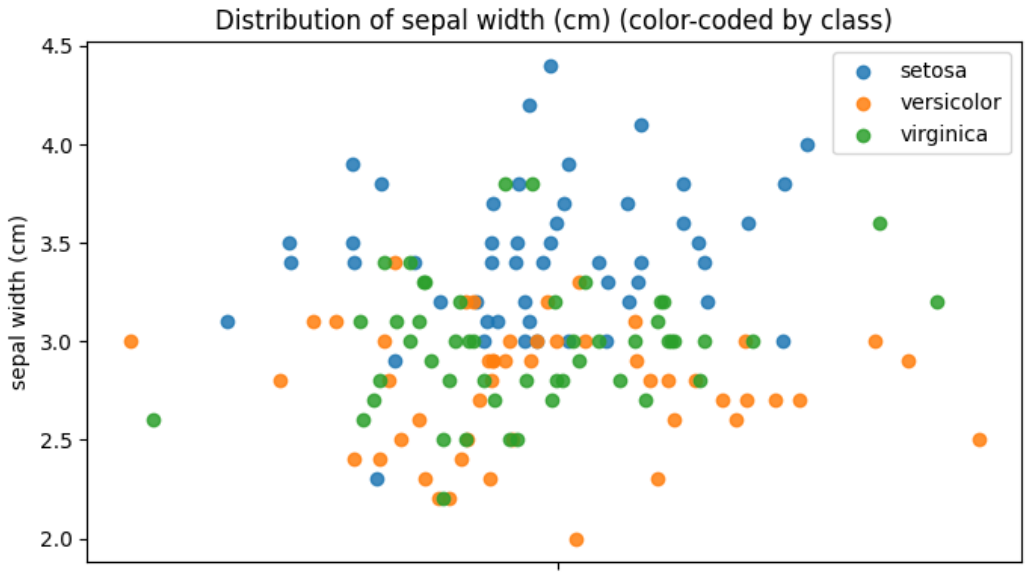

Figure 5. Distribution of Sepal Width (cm), color-coded by class.

Analysis of Visualizations

Feature Distribution Implications

The feature analysis plots show that petal length and width are the best features to differentiate iris classes. For example, the setosa class occupies a low range for both petal length and petal width; however, the versicolor and virginica classes occupy higher ranges with some overlap. In contrast, sepal length and sepal width exhibit more class overlapping, especially between versicolor and virginica. The described distribution pattern indicates that petal-based measurements serve as major discriminators, while sepal-based measurements provide additional context.

Observed Patterns and Analytical Implications

The correlation heatmap announces a very strong positive relationship between petal length and petal width, and a moderately strong relationship between petal length and sepal length. These relationships suggest overlap between certain measurements, making it more difficult to interpret individual coefficients in linear models. Also, the feature analysis plots corroborate this, showing tighter clustering for setosa and broader scatter for versicolor and virginica, indicative of differing variance across classes. Ultimately, petal-based measurements are the dataset's strongest predictors; however, given their strong positive relationship, interpreting coefficients is more complex.

Linear Regression Outputs

Case 1 (30% Samples)

- **Input feature order:** [sepal length (cm), sepal width (cm), petal width (cm)]
- **Slopes (coef_):** [0.6950645985019033, -0.6486869946240046, 1.4833247991441318]
- **Intercept:** -0.099447
- **Chosen excluded sample index:** 73
- **Actual petal length:** 4.700000
- **Predicted petal length:** 4.104114
- **RMSE on test set:** 0.320953

Case 2 (80% Samples)

- **Input feature order:** [sepal length (cm), sepal width (cm), petal width (cm)]
- **Slopes (coef_):** [0.7228146259066683, -0.6358164939643193, 1.4675240315042082]
- **Intercept:** -0.262196
- **Chosen excluded sample index:** 73
- **Actual petal length:** 4.700000
- **Predicted petal length:** 4.127716
- **RMSE on test set:** 0.360578

Model Comparison and Conclusion

With 30% samples, testing yielded an RMSE of 0.320953. In contrast, with 80% samples, testing yielded an RMSE of 0.360578. Typically, RMSE measures prediction error on testing data. Therefore, the Linear Regression model demonstrates better generalization performance with the selected random seed and at 30% samples. Notably, larger training sets often improve parameter stability, but evaluation must rely on the selected metric rather than initial expectation. Simply, the 0.3 training split provided the more accurate predictive model for petal length.

Appendix

```
import numpy
import pandas
import matplotlib.pyplot
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error


def loadIrisDatasetAsFriendlyTables():
    irisDatasetBundle = load_iris(as_frame=True)

    flowerMeasurementTable = irisDatasetBundle.data
    flowerTypeNumberLabels = irisDatasetBundle.target

    flowerTypeNameLabels = flowerTypeNumberLabels.map({
        0: "setosa",
        1: "versicolor",
        2: "virginica"
    })

    return flowerMeasurementTable, flowerTypeNameLabels


def drawCorrelationHeatmapForAllMeasurements(flowerMeasurementTable):
    correlationCoefficientTable = flowerMeasurementTable.corr()

    figurePaper, drawingArea = matplotlib.pyplot.subplots(figsize=(7, 5))

    heatmapPicture = drawingArea.imshow(
        correlationCoefficientTable.to_numpy(),
        interpolation="nearest"
    )

    drawingArea.set_title("Iris Measurement Correlation Heatmap")
```

```
    featureNameList = list(correlationCoefficientTable.columns)
    featureCount = len(featureNameList)
    featureIndexList = list(range(featureCount))

    drawingArea.set_xticks(featureIndexList)
    drawingArea.set_yticks(featureIndexList)

    drawingArea.set_xticklabels(featureNameList, rotation=45, ha="right")
    drawingArea.set_yticklabels(featureNameList)

    figurePaper.colorbar(
        heatmapPicture,
        ax=drawingArea,
        label="Correlation coefficient"
    )

    matplotlib.pyplot.tight_layout()
    matplotlib.pyplot.show()


def drawOneFeatureDotPlotPerMeasurementColorCodedByFlowerType(
    flowerMeasurementTable,
    flowerTypeNameLabels
):
    repeatableTinyWiggleMaker = numpy.random.default_rng(0)

    flowerTeamNameList = ["setosa", "versicolor", "virginica"]

    for measurementName in flowerMeasurementTable.columns:
        matplotlib.pyplot.figure(figsize=(7, 4))

        for flowerTeamName in flowerTeamNameList:
            rowsThatBelongToThisFlowerTeam = (flowerTypeNameLabels == flowerTeamName)

            measurementValuesForThisFlowerTeam = flowerMeasurementTable.loc[
                rowsThatBelongToThisFlowerTeam,
                measurementName
```

```
            ].to_numpy()

            verticalDotStripCenterLine = 0.0

            tinySideToSideWigglesSoDotsDoNotStack = repeatableTinyWiggleMaker.normal(
                loc=0.0,
                scale=0.03,
                size=len(measurementValuesForThisFlowerTeam)
            )

            dotHorizontalPositions = verticalDotStripCenterLine + tinySideToSideWigglesSoDotsDoNotStack

            matplotlib.pyplot.scatter(
                dotHorizontalPositions,
                measurementValuesForThisFlowerTeam,
                label=flowerTeamName,
                alpha=0.85
            )

        matplotlib.pyplot.title(f"Distribution of {measurementName} (color-coded by class)")
        matplotlib.pyplot.xticks([0], [""])
        matplotlib.pyplot.ylabel(measurementName)
        matplotlib.pyplot.legend()
        matplotlib.pyplot.tight_layout()
        matplotlib.pyplot.show()


def runLinearRegressionExperimentToPredictPetalLength(
    flowerMeasurementTable,
    trainingPortionSize,
    repeatableShuffleSeed=42
):
    columnNameToPredict = "petal length (cm)"

    realPetalLengthAnswers = flowerMeasurementTable[columnNameToPredict].to_numpy()

    inputCluesWithoutPetalLengthSoNoCheating = flowerMeasurementTable.drop(
```

```
    columns=[columnNameToPredict]
).to_numpy()

originalRowIndexStickers = numpy.arange(len(flowerMeasurementTable))

(
    trainingClues,
    testingClues,
    trainingAnswers,
    testingAnswers,
    trainingRowIndexStickers,
    testingRowIndexStickers
) = train_test_split(
    inputCluesWithoutPetalLengthSoNoCheating,
    realPetalLengthAnswers,
    originalRowIndexStickers,
    train_size=trainingPortionSize,
    random_state=repeatableShuffleSeed,
    shuffle=True
)

linearRegressionModel = LinearRegression()
linearRegressionModel.fit(trainingClues, trainingAnswers)

predictedPetalLengthsForAllTestingFlowers = linearRegressionModel.predict(testingClues)

testRootMeanSquaredError = float(
    numpy.sqrt(
        mean_squared_error(testingAnswers, predictedPetalLengthsForAllTestingFlowers)
    )
)

chosenExcludedTestFlowerPosition = 0

chosenExcludedOriginalRowIndex = int(testingRowIndexStickers[chosenExcludedTestFlowerPosition])
chosenExcludedActualPetalLength = float(testingAnswers[chosenExcludedTestFlowerPosition])
```

```
    chosenExcludedCluesAsSingleRow = testingClues[chosenExcludedTestFlowerPosition].reshape(1, -1)
    chosenExcludedPredictedPetalLength = float(
        linearRegressionModel.predict(chosenExcludedCluesAsSingleRow)[0]
    )

    slopesForEachInputFeature = linearRegressionModel.coef_.tolist()
    interceptStartingPoint = float(linearRegressionModel.intercept_)

    return {
        "trainingPortionSize": trainingPortionSize,
        "slopesForEachInputFeature": slopesForEachInputFeature,
        "interceptStartingPoint": interceptStartingPoint,
        "chosenExcludedOriginalRowIndex": chosenExcludedOriginalRowIndex,
        "chosenExcludedActualPetalLength": chosenExcludedActualPetalLength,
        "chosenExcludedPredictedPetalLength": chosenExcludedPredictedPetalLength,
        "testRootMeanSquaredError": testRootMeanSquaredError,
    }


def main():
    flowerMeasurementTable, flowerTypeNameLabels = loadIrisDatasetAsFriendlyTables()
    drawCorrelationHeatmapForAllMeasurements(flowerMeasurementTable)

    drawOneFeatureDotPlotPerMeasurementColorCodedByFlowerType(
        flowerMeasurementTable,
        flowerTypeNameLabels
    )

    for trainingPortionSize in (0.3, 0.8):
        experimentResult = runLinearRegressionExperimentToPredictPetalLength(
            flowerMeasurementTable,
            trainingPortionSize=trainingPortionSize
        )

        print("Input feature order: [sepal length (cm), sepal width (cm), petal width (cm)]")
        print(f"Slopes (coef_): {experimentResult['slopesForEachInputFeature']}")
        print(f"Intercept: {experimentResult['interceptStartingPoint']:.6f}")
```

```
    print(f"Chosen excluded sample index: {experimentResult['chosenExcludedOriginalRowIndex']}")
    print(f"Actual petal length: {experimentResult['chosenExcludedActualPetalLength']:.6f}")
    print(f"Predicted petal length: {experimentResult['chosenExcludedPredictedPetalLength']:.6f}")
    print(f"RMSE on test set: {experimentResult['testRootMeanSquaredError']:.6f}")


if __name__ == "__main__":
    main()
```